Project Mario Bros

Authors

Diego Madroñal de Mesa

Sonia Andreea Spranceana

9/12/2025 Group 97

Index

Introduction …….3

Instructions ……..3

Class design …….....4

Relevant methods and algorithms ……...5

- Main script
- Conveyor script
- Package handling
- Input cooldowns
- Pause and restart functions
- Using formulas for sprites and packages

Game user manual ……...7

Conclusions …….....9

Introduction

In this version of Mario Bros, Mario and Luigi work together in a packages factory. Their mission is to ensure that packages arrive correctly at the delivery truck without falling to the ground. The Boss punishes mistakes, while floors, stairs and truck mechanics create additional gameplay challenges.

This document presents the main characteristics of the game and its code, aiming to make understandable the parts that may be difficult to comprehend.

Instructions

In order to play the game, download the ZIP file and extract it into your desired folder. Make sure you have Python >= 3.10 installed and run the following command to install the project dependencies:

pip install pixel

Open the project in your desired editor and run the script *main.py* to enter the game. If you wish to edit or look at the sprites, they are found in the assets folder and can be viewed/edited with the command:

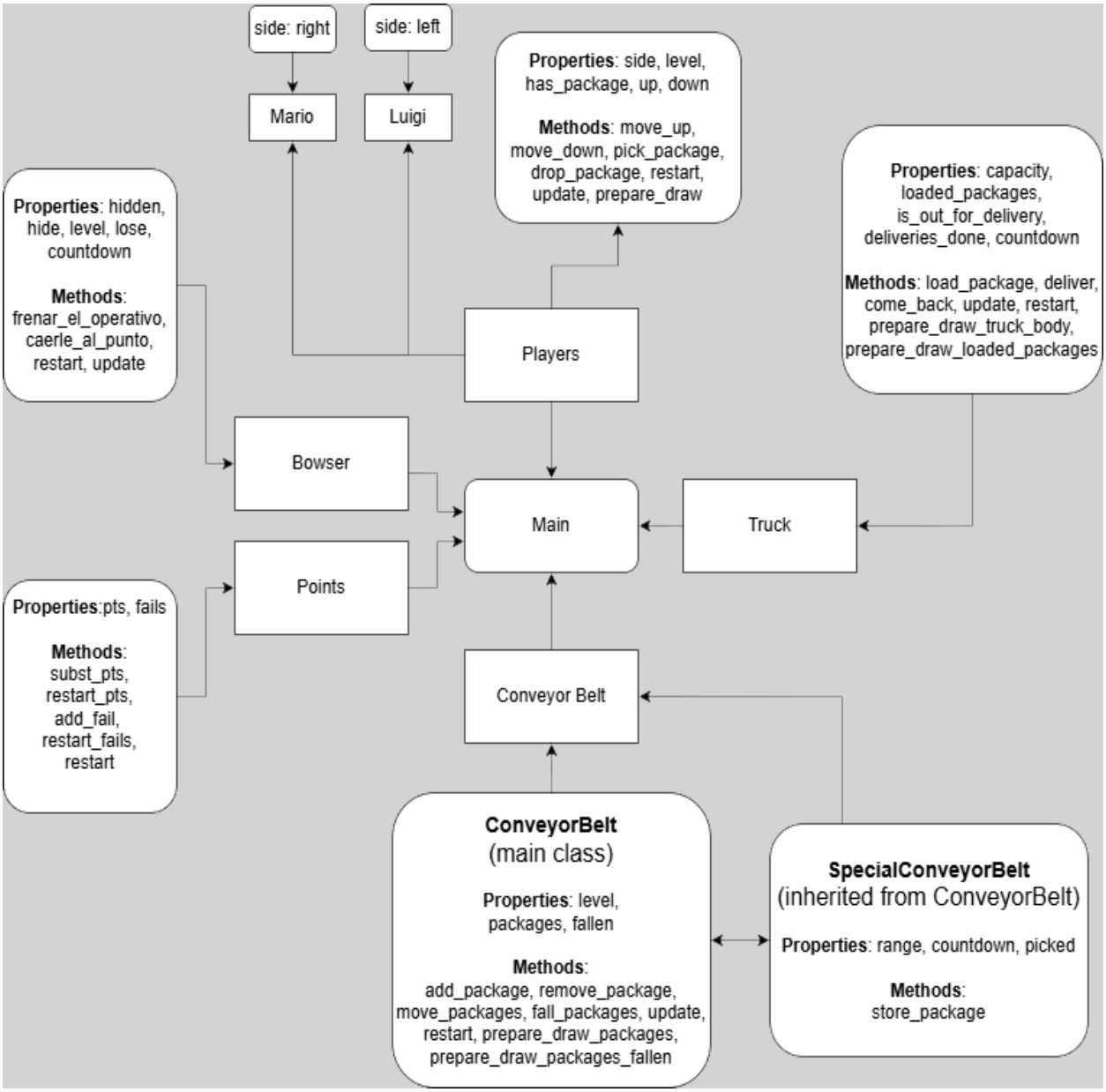
pyxel edit sprites

If for any reason this command gives you trouble, try using

py -m pyxel edit sprites

The game can be played as much as you want without having to run the program again, to do this, just make sure to keep the window open and press *"R"* to play again. This achieves the same result as closing and reopening it, but in a far more convenient way.

Class Design



The diagram shows the main classes used, with all their properties, methods and inheritances.

Although the different classes use “private attributes”, it’s important to note that Python does not truly support public, nor private attributes. The double underscore (“__”) before an attribute triggers name mangling. Therefore, while the attribute is intended to be private, it can still be accessed from outside the class using “*_Class_Attribute”.* Obviously, doing so is not recommended, but it is useful to remember that it’s possible.

Each class contains at least one update method and one draw method. This design keeps the *‘__main__”* section cleaner. Instead of calling every individual method that needs updating, we simply call “*object.update*”, which handles calling the other functions and user inputs (for Mario and Luigi). Likewise, the draw method controls the visual display, although in some cases the *object.draw* returns a list (e.g ConveyorBelt), which we process using *pyxel.draw (*object.draw)*.

Most relevant methods and algorithms

Main script (main.py)

The main script contains the *App* class, which is responsible for handling the game loop. This is not the script in which most of the logic happens, but it is the one that calls all the relevant methods defined in the other classes.

The *draw()* method is the one that loads the Pyxel textures. In order to improve performance, we bundled the textures as much as possible (some textures had to be drawn separately, but a large amount of them were fit into a single sprite).

Conveyor script (conveyorbelt.py)

This is the script where the majority of the logic and calculations are handled. This script interacts with all the other classes, since it is responsible for the position of the packages, which trigger the most important game events (points update, fails, truck loading).

In order to make the gameplay smooth and the difficulty appropriate, the conveyors update every second game tick, rather than on every single tick.

Package handling

One of the most unique design decisions made during the development of the game can be found in the package handling. Packages are not a "separated" object; instead, they are managed in the *ConveyorBelt* and *SpecialConveyorBelt* classes, as well as in the *Character Class*.

This approach allows us to create a far less resource-consuming videogame, as it avoids creating new objects or tracking information (every package memory location or position on the board).
Instead, packages are represented as 1s and 0s within the aforementioned classes. We believe that this design is not only much more optimized but also smarter and more innovative.

Furthermore, it is worth noting that there is an invisible *SpecialConveyorBelt* that exists between the Truck and Luigi. Once again, this was a deliberate decision that allows us to manage packages in a more structured and logical way, providing a well-rounded experience for the players and maintaining the consistent structure and rhythm of the game.

Input cooldown

Mario and Luigi can't move faster than a set speed; that is, a player's input won't be read, nor processed until a certain amount of time has passed since the character's previous input (Mario's input doesn't affect Luigi's). This way, not only do we provide a much more faithful experience to the original arcade games but also adds an extra layer of

difficulty, while giving the game a more professional feel (what is known in the game industry as *"juice").*

Pause and restart functions

The game features a Pause button ("P") and a Restart button ("R"), that can be used to play again after reaching 3 failures. Pressing the restart button resets the score and fails, returns Mario and Luigi to their initial position and removes all packages from the screen.

Using formulas for sprites and packages

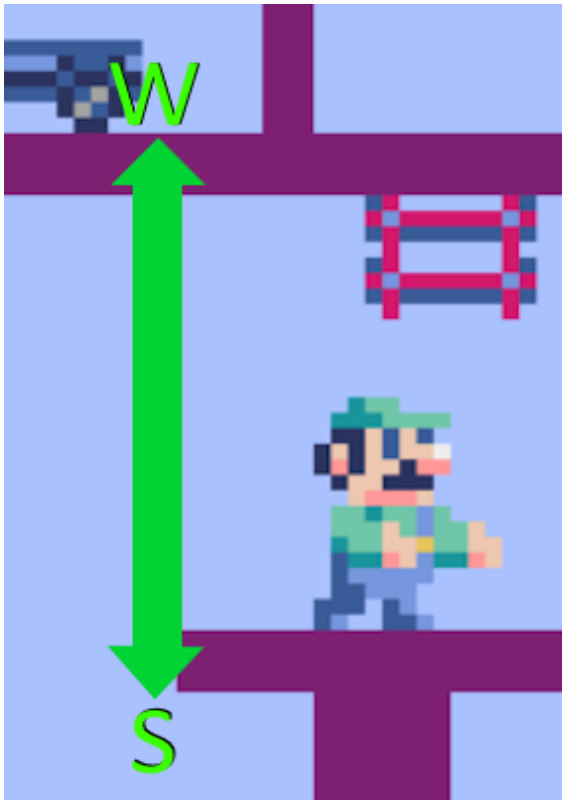
Most of the sprites' positions are calculated via polynomials. This way, the game is much more scalable, and any tweak can be done in a general way instead of replacing a long list of coordinates.

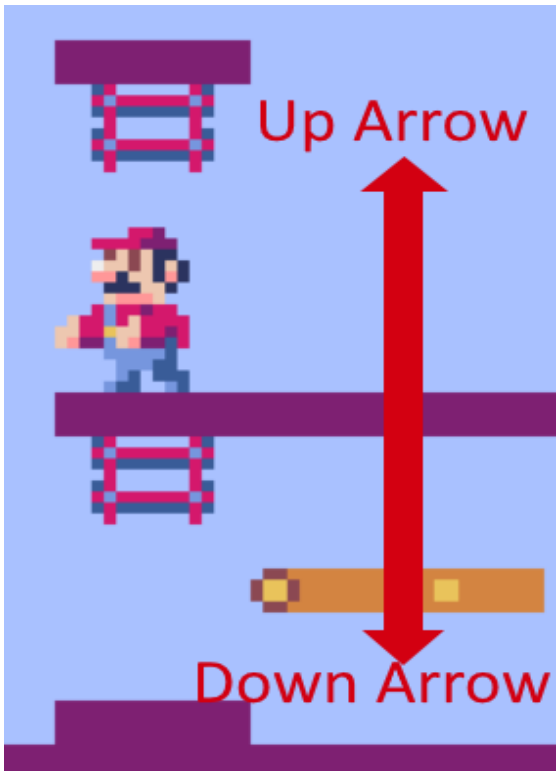
We have used as many formulas as we could to generalize different cases. This design makes the code much easier to modify in the future without requiring a lot of rewrites. An example for this can be found in the *ConveyorBelt* class, where *character.level* is checked to add or drop packages. Different polynomials, such as "2*self.level +2", are used for the package handling logic.

Game user manual

The main purpose of the player is to load as many of the incoming crates as possible onto the delivery truck. The crates will slowly approach the players, starting from Mario's side.

The player must control Mario and Luigi, using the up-down arrow keys and the W-S keys respectively.


W
S


Up Arrow
Down Arrow

The characters must line up with the incoming crates to load them onto the next container. If either one of them fails to catch a package, the crate falls and Bowser pops up, adding a fail to the fail count. The game ends after 3 fails.

Whenever the player manages to load 8 crates, the truck delivers them and the score goes up. While it may seem very simple, the game is quite difficult. Try delivering as many crates as possible!

The player can pause the game at any moment by pressing “P” and unpause by pressing again. A “Paused” white text will appear to inform you and until you unpause Mario and Luigi won’t be able to move, and the packages position won’t change.

Conclusions

This project has been a good exercise in the use of Object-Oriented Programming concepts in a practical setting: game development. By recreating the mechanics of a classic game, we worked with the theoretical concepts we learned about this semester and used them in a real project. The 80s theme presented an interesting challenge, since we had to make the gameplay feel smooth and engaging with the limitation of the arcade style.

Pyxel was an important tool for the development of our project. It was not difficult to learn and it provided us all the tools needed for a simple retro game. We created some of our own sprites and used textures created by other students, resulting in a visually pleasing design.

We succeeded in creating a fun game that captures the aesthetic and atmosphere of the 80’s, while using algorithms and programming techniques that prioritize optimization and scalability (e.g. check how each position in the screen is calculated).